CS-302 Artificial Intelligence Experiment # 09

Understanding of Hill Climbing Search and A Star Search

Objective/s:

- What is Hill Climbing Search and A Star Search
- Implementation of Hill Climbing Search and A star Search
- Lab Tasks

Theory:

Hill climbing Search:

Hill climbing is a simple optimization algorithm used to find the best possible solution to a problem among a set of candidate solutions. It's named because it works by analogy with climbing a hill, where the aim is to reach the peak (the best solution) by continually moving in the direction of increasing elevation (improved solution).

Algorithm:

1. Initialize: Start with a random solution.
2. Evaluate: Calculate the value of the current solution using an objective function. This function measures how good or bad the solution is.
3. Generate neighbors: Create neighboring solutions by making small changes to the current solution.
4. Select the best neighbor: Among the neighboring solutions, select the one that improves the objective function the most.
5. Repeat: Set the current solution to the best neighbor found and repeat steps 2-4 until either a stopping condition is met (such as reaching a maximum number of iterations or a solution with satisfactory performance) or there are no better neighbors.

Hill climbing has its limitations, such as getting stuck in local optima (situations where there are better solutions, but they are not directly accessible from the current solution). To address this, variations of hill climbing algorithms like simulated annealing and genetic algorithms introduce randomness or more complex mechanisms to explore a broader solution space.

A Star Search:

The A* search algorithm is a popular and widely used pathfinding algorithm in computer science. It's particularly useful for finding the shortest path between nodes in a graph, such as in navigation systems or game AI.

Algorithm:

1. Initialization: Start with an initial node (the starting point) and add it to the "open set." Also, initialize the cost of reaching this node from the start (g-value) and the estimated cost to reach the goal from this node (h-value).
2. Expansion: Repeat the following steps until either the goal node is reached or the open set is empty:
    - Select the node from the open set with the lowest total cost (f-value), where f(n) = g(n) + h(n). This node is considered the current node.
    - If the current node is the goal node, stop; you have found the shortest path.
    - Otherwise, remove the current node from the open set and add it to the "closed set" (to mark it as visited).
    - Expand the current node by considering all its neighbors (adjacent nodes) and calculate their tentative g-value (the cost to reach them from the start) and h-value (the estimated cost to reach the goal from them).
    - If a neighbor is not in the open set, add it and set its g-value and h-value. If it is in the open set but the new g-value is lower than its previous one, update its g-value and re-calculate its f-value.
3. Termination: If the open set becomes empty before reaching the goal node, then there is no path from the start node to the goal node, indicating that no solution exists.

Lab Task:

Implement the A* search algorithm to find the shortest path between two points on a grid.

Task Description:

1. **Setup**: Create a grid environment where each cell can either be traversable (empty) or blocked (obstacle).
2. **Inputs**:
    - Grid size (rows and columns).
    - Starting point coordinates (row and column).
    - Goal point coordinates (row and column).

- Obstacle locations (list of coordinates).

3. **Algorithm Implementation**:
   - Implement the A* search algorithm to find the shortest path from the starting point to the goal point while avoiding obstacles.
   - Use Manhattan distance (or any other appropriate heuristic) as the heuristic function for estimating the distance from each cell to the goal.
4. **Output**:
   - The shortest path from the starting point to the goal point.
   - Visualization of the grid with the path highlighted.